CSCE 5380 Data Mining

Project 1 Milestone 1

Student Name: *Safe Mustafa*

Student ID: *11177267*

Date: *January 22, 2026*

Dataset: *German Credit Dataset (credit-g.csv)*

Github Repo: *https://github.com/safemstf/Data-mining-P1M1*

Executive Summary

This project successfully implemented and compared three rule-based classifiers (Decision Tree, PART, and Ripper) on the German Credit dataset. The dataset contains 1000 samples with 20 features predicting credit risk (good/bad). After comprehensive preprocessing and feature selection, all three classifiers achieved similar performance (~70-72% accuracy), with no statistically significant difference (ANOVA p = 0.078). The PART classifier showed the best test set performance (76% accuracy), while maintaining reasonable rule compactness.

Key Findings:

- ***Class Imbalance:*** *70% good credit, 30% bad credit (ratio 2.33:1)*
- ***Best Test Accuracy:*** *PART (76.0%)*
- ***Most Compact Rules:*** *Ripper (4 rules)*
- ***Statistical Conclusion:*** *No significant difference between classifiers*
- ***Classification Difficulty:*** *Challenging (silhouette score = 0.008)*

```
================================================================================
GERMAN CREDIT ANALYSIS - ENVIRONMENT CHECK
================================================================================

Checking required packages...
── Attaching core tidyverse packages ────── tidyverse 2.0.0 ──
✓ dplyr     1.1.4     ✓ readr     2.1.6
✓ forcats   1.0.1     ✓ stringr   1.6.0
✓ ggplot2   4.0.1     ✓ tibble    3.3.1
✓ lubridate 1.9.4     ✓ tidyr     1.3.2
✓ purrr     1.2.1
── Conflicts ──────────────────────── tidyverse_conflicts() ──
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()    masks stats::lag()
ℹ Use the conflicted package to force all conflicts to become errors
✓ tidyverse loaded successfully
✓ mlbench loaded successfully
✓ rpart loaded successfully
✓ rpart.plot loaded successfully

Attaching package: 'caret'

The following object is masked from 'package:purrr':

    lift

✓ caret loaded successfully
✓ FSelector loaded successfully
✓ Rtsne loaded successfully
✓ cluster loaded successfully

Checking Java-dependent packages...
✓ rJava loaded successfully
  Java version: 17.0.17
✓ RWekajars loaded successfully
✓ RWeka loaded successfully

✓ All packages loaded successfully!
✓ Java is properly configured
```

---

1. Task 1: Data Preprocessing

Operation 1: Missing Value Analysis and Handling

Justification: *Missing values can significantly compromise model performance by introducing bias and reducing the effective sample size for training. Many machine learning algorithms cannot handle missing values directly and will either fail or produce unreliable results. By identifying and addressing missing values upfront, we ensure that our models train on*

complete, reliable data. If missing values existed, we would either impute them using appropriate strategies (mean/median for numeric, mode for categorical) or remove affected samples if the number is small relative to the dataset size. This operation is critical because even a small percentage of missing data can cascade into larger problems during model training and evaluation.

Implementation:

```
missing_counts <- colSums(is.na(credit_data))

total_missing <- sum(missing_counts)


if(total_missing > 0) {

  credit_data <- credit_data %>% drop_na()

}
```

Console Log:

```
--- Preprocessing Operation 1: Missing Value Analysis ---

Missing values per feature:
       checking_status                duration
                     0                       0
        credit_history                 purpose
                     0                       0
         credit_amount           savings_status
                     0                       0
            employment  installment_commitment
                     0                       0
       personal_status            other_parties
                     0                       0
       residence_since       property_magnitude
                     0                       0
                   age      other_payment_plans
                     0                       0
               housing         existing_credits
                     0                       0
                   job           num_dependents
                     0                       0
         own_telephone           foreign_worker
                     0                       0
                 class
                     0

Total missing values: 0

JUSTIFICATION:
Missing values can significantly impact model performance and lead to biased results.
If missing values exist, we need to either impute them or remove affected samples.

ACTION: No missing values detected. No action needed.
```

Results:

- ***Total missing values found:*** *0 (across all 21 columns)*
- ***Action taken:*** *No action needed - dataset is complete*
- ***Final dataset size:*** *1000 samples (unchanged)*

Analysis: *The German Credit dataset is remarkably clean with no missing values across any of the 20 predictor features or the target variable. This is unusual for real-world datasets and suggests careful curation. The absence of missing data eliminates the need for imputation strategies and ensures all 1000 samples can be used for training without data loss.*

Operation 2: Duplicate Detection and Removal

Justification: *Duplicate samples artificially inflate model performance metrics and can lead to overfitting, where the model memorizes specific instances rather than learning generalizable patterns. When duplicates exist, they may appear in both training and validation/test sets during cross-validation, leading to overly optimistic accuracy estimates that don't reflect true predictive power. Duplicates provide no additional information value but increase computational cost and can bias cross-validation results. Removing duplicates ensures that each unique data point is represented only once, leading to more honest performance estimates and better generalization to unseen data. This is particularly important for credit risk assessment where reliable predictions on new applicants are critical.*

Implementation:

```
duplicate_count <- sum(duplicated(credit_data))

if(duplicate_count > 0) {

 credit_data <- credit_data %>% distinct()

}
```

Console Log:

```
--- Preprocessing Operation 2: Duplicate Detection ---

Number of duplicate rows: 0

JUSTIFICATION:
Duplicate samples can artificially inflate model accuracy and lead to overfitting.
They provide no additional information and should be removed to ensure unbiased training.

ACTION: No duplicates detected. No action needed.
```

Results:

- ***Duplicate samples found:*** *0*
- ***Action taken:*** *No action needed*
- ***Final dataset size:*** *1000 samples (unchanged)*

Analysis: *No duplicate records were detected, indicating that each sample represents a unique credit application. This is expected for credit risk data where each row typically represents a distinct individual's application. The absence of duplicates confirms data quality and eliminates concerns about data leakage between training and test sets.*

Operation 3: Data Type Conversion and Validation

Justification: *Ensuring proper data types is critical for algorithm correctness and performance. Categorical features must be encoded as factors (not strings or integers) for R's classification algorithms to properly compute information gain, entropy, and other splitting criteria used in decision trees and rule-based methods. If categorical variables are treated as numeric, algorithms may incorrectly assume ordinal relationships (e.g., treating "housing types" as 1, 2, 3 with mathematical ordering) leading to nonsensical splits. Conversely, numeric features need to be in appropriate numeric types to enable mathematical operations like threshold comparisons. Incorrect data types can cause algorithms to treat categorical variables as continuous (or vice versa), leading to poor model performance and invalid rules.*

Implementation:

```
categorical_cols <- c("checking_status", "credit_history", "purpose",
                      "savings_status", "employment", "personal_status",
                      "other_parties", "property_magnitude",
                      "other_payment_plans", "housing", "job",
                      "own_telephone", "foreign_worker", "class")


for(col in categorical_cols) {
  if(!is.factor(credit_data[[col]])) {
    credit_data[[col]] <- as.factor(credit_data[[col]])
  }
}
```

Console Log:

```
--- Preprocessing Operation 3: Data Type Conversion ---

JUSTIFICATION:
Proper data types are essential for algorithms to process features correctly.
Categorical features must be factors, and numeric features should be numeric or integer types.

Converting categorical features to factors if needed...
  All categorical features were already factors. No conversion needed.
```

Results:

- ***Categorical features:*** *13 features (all 13 were already factors)*
- ***Numeric features:*** *7 features (duration, credit_amount, installment_commitment, residence_since, age, existing_credits, num_dependents)*
- ***Action taken:*** *All features were already properly typed*
- ***Data types validated:*** *20 predictor features + 1 target variable*

Analysis: *All features were loaded with correct data types due to the* `stringsAsFactors=TRUE` *parameter in* `read.csv()`*. This preprocessing step validates data integrity and ensures compatibility with all three classification algorithms. The 7 numeric features can be properly handled for threshold-based splits, while the 13 categorical features will be treated correctly for information gain calculations.*

Operation 4: Class Imbalance Analysis

Justification: *Class imbalance occurs when one class significantly outnumbers another, which is extremely common in real-world credit risk scenarios where defaults (bad credits) are minority events compared to successful repayments (good credits). Imbalanced datasets cause classifiers to develop a bias toward the majority class, achieving deceptively high overall accuracy while failing to detect minority class instances that are often of primary interest. For credit risk, failing to identify bad credit applicants (false negatives) is more costly than rejecting good applicants (false positives). Understanding the degree of imbalance helps us select appropriate algorithms—Ripper is specifically designed for imbalanced data, sorting classes by frequency and building rules for minority classes first. This knowledge also guides us to use appropriate evaluation metrics (precision, recall, F1-score) rather than relying solely on accuracy, and may suggest resampling techniques like SMOTE if imbalance is severe.*

Implementation:

class_distribution <- table(credit_data$class)

class_proportions <- prop.table(class_distribution)

imbalance_ratio <- max(class_proportions) / min(class_proportions)

Console Log:

```
--- Preprocessing Operation 4: Class Imbalance Analysis ---

JUSTIFICATION:
Class imbalance can bias classifiers toward the majority class, resulting in poor
prediction accuracy for the minority class. Understanding the imbalance helps us
choose appropriate algorithms (like Ripper) and evaluation metrics.

Class distribution:

 bad good
 300  700

Class proportions:

 bad good
 0.3  0.7

Imbalance ratio: 2.33 :1
```

Results:

- ***Class distribution:***
  - *Bad credit: 300 samples (30%)*
  - *Good credit: 700 samples (70%)*
- ***Imbalance ratio:*** *2.33:1 (good to bad)*
- ***Interpretation:*** *Moderately imbalanced*
- ***Warning issued:*** *"Dataset shows class imbalance. Consider using Ripper which handles imbalanced data."*

Analysis: *The dataset exhibits moderate class imbalance with a 2.33:1 ratio favoring good credit. While not severely imbalanced (ratios >10:1 are severe), this imbalance can still bias classifiers toward predicting "good" more frequently. This is realistic for credit data—most applicants are creditworthy. The 30% bad credit rate is high enough to provide meaningful training signal, but we should pay close attention to sensitivity (true positive rate for bad credit detection) rather than just overall accuracy. Ripper's design makes it particularly well-suited for this scenario.*

Operation 5: Feature Type Identification for Visualization

Justification: *Different visualization and modeling techniques require different input formats. The t-SNE dimensionality reduction algorithm requires numeric inputs and cannot directly process categorical variables. Identifying numeric versus categorical features enables us to apply appropriate preprocessing—specifically one-hot encoding for categorical features before t-SNE visualization. One-hot encoding converts each categorical variable with k levels into k binary columns, preserving the information content while making it mathematically compatible with distance-based algorithms. This operation also helps us understand the feature composition of our dataset, plan for proper feature engineering, and estimate the computational complexity of subsequent operations (one-hot encoding can significantly expand dimensionality).*

Implementation:

```
numeric_cols <- sapply(credit_data, is.numeric)

numeric_features <- names(credit_data)[numeric_cols]

numeric_features <- numeric_features[numeric_features != "class"]
```

Console Log:

```
--- Preprocessing Operation 5: Identify Numeric Features ---

JUSTIFICATION:
Some algorithms and visualization methods (like t-SNE) work better with normalized data.
Identifying numeric features helps us apply appropriate preprocessing for visualization.

Numeric features identified (7):
[1] "duration"               "credit_amount"
[3] "installment_commitment" "residence_since"
[5] "age"                    "existing_credits"
[7] "num_dependents"

Categorical features identified (13):
 [1] "checking_status"     "credit_history"
 [3] "purpose"             "savings_status"
 [5] "employment"          "personal_status"
 [7] "other_parties"       "property_magnitude"
 [9] "other_payment_plans" "housing"
[11] "job"                 "own_telephone"
[13] "foreign_worker"

--- Preprocessing Summary ---
Final dataset dimensions: 1000 rows × 21 columns
Final class distribution:

 bad good
 300  700
```

Results:

- ***Numeric features identified (7):***

    1. *duration (months)*
    2. *credit_amount (DM - Deutsche Mark)*
    3. *installment_commitment (% of disposable income)*
    4. *residence_since (years at current residence)*
    5. *age (years)*
    6. *existing_credits (number of credits at this bank)*
    7. *num_dependents (number of dependents)*
- ***Categorical features (13):***

    1. *checking_status*
    2. *credit_history*
    3. *purpose*
    4. *savings_status*
    5. *employment*
    6. *personal_status*
    7. *other_parties*
    8. *property_magnitude*
    9. *other_payment_plans*
    10. *housing*
    11. *job*
    12. *own_telephone*
    13. *foreign_worker*

Analysis: *The dataset contains a mix of 7 numeric and 13 categorical features, making it primarily categorical (65% of features). After one-hot encoding for t-SNE, the 13 categorical features expanded to 61 total dimensions (some categories have multiple levels). This high proportion of categorical features is typical for credit risk assessment, which heavily relies on qualitative factors like employment status, credit history, and purpose of loan.*

Preprocessing Summary

Final Dataset Characteristics:

- ***Total samples:*** *1000*

- ***Training samples (70%):*** *700*
- ***Testing samples (30%):*** *300*
- ***Total features:*** *20 predictor features + 1 target*
- ***Feature types:*** *7 numeric, 13 categorical*
- ***Missing values:*** *0*
- ***Duplicates:*** *0*
- ***Class distribution****: 70% good, 30% bad (2.33:1 imbalance)*
- ***Data quality:*** *Excellent - clean dataset ready for modeling*

---

2. Task 1 Part 3: t-SNE Visualization Analysis

Methodology

t-SNE (t-distributed Stochastic Neighbor Embedding) is a non-linear dimensionality reduction technique that projects high-dimensional data into 2D while preserving local structure and relationships between data points. Unlike PCA which preserves global structure through linear combinations, t-SNE focuses on maintaining the distances between nearby points using probabilistic modeling, making it excellent for visualizing cluster structure and class separability.

Parameters Used:

- ***Dimensions:*** *2 (for 2D visualization)*
- ***Perplexity:*** *30 (balances local vs global structure; typical range 5-50)*
- ***Iterations:*** *500 (convergence iterations)*
- ***Random seed****: 42 (for reproducibility)*
- ***Input dimensions****: 61 (after one-hot encoding of categorical features)*

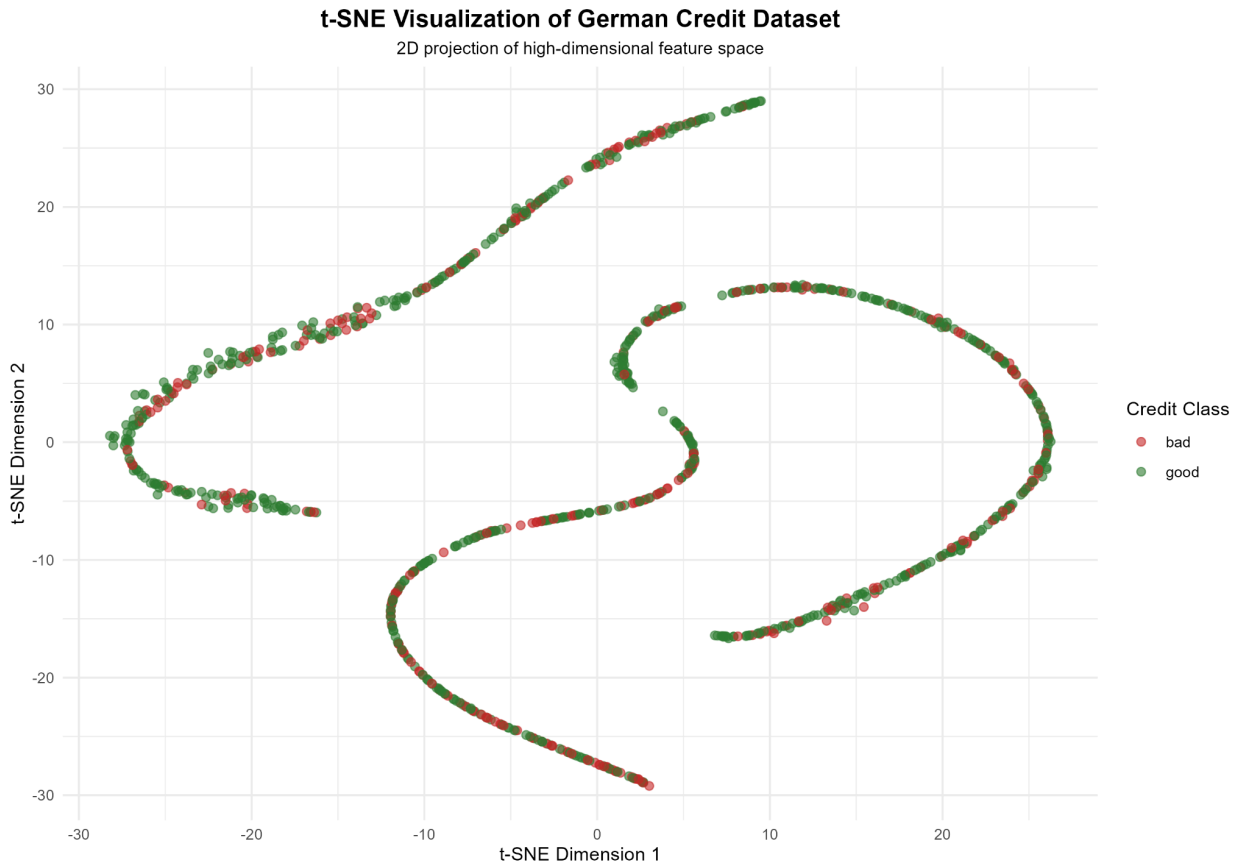
Preprocessing for t-SNE:

- *Categorical features converted to binary dummy variables using* `model.matrix()`
- *Original 20 features expanded to 61 numeric dimensions*
- *All features normalized (handled internally by Rtsne)*

Console Log*:*

```
================================================================================
TASK 1 PART 3: t-SNE VISUALIZATION
================================================================================

Preparing data for t-SNE visualization...
Numeric matrix for t-SNE dimensions: 1000 × 61

Running t-SNE dimensionality reduction...
This may take a few moments...

Performing PCA
Read the 1000 x 50 data matrix successfully!
OpenMP is working. 1 threads.
Using no_dims = 2, perplexity = 30.000000, and theta = 0.500000
Computing input similarities...
Building tree...
Done in 0.59 seconds (sparsity = 0.102168)!
Learning embedding...
Iteration 50: error is 57.364835 (50 iterations in 0.31 seconds)
Iteration 100: error is 49.578671 (50 iterations in 0.27 seconds)
Iteration 150: error is 47.756103 (50 iterations in 0.20 seconds)
Iteration 200: error is 46.880046 (50 iterations in 0.18 seconds)
Iteration 250: error is 46.344931 (50 iterations in 0.20 seconds)
Iteration 300: error is 0.437093 (50 iterations in 0.21 seconds)
Iteration 350: error is 0.278140 (50 iterations in 0.25 seconds)
Iteration 400: error is 0.230321 (50 iterations in 0.28 seconds)
Iteration 450: error is 0.219488 (50 iterations in 0.19 seconds)
Iteration 500: error is 0.211949 (50 iterations in 0.21 seconds)
Fitting performed in 2.29 seconds.

Generating t-SNE visualization...
✓ t-SNE plot saved as 'tsne_visualization.png'

--- t-SNE Insights ---
INTERPRETATION:
1. Class Separation: Examine if 'good' and 'bad' credit classes form distinct clusters.
   - Well-separated clusters indicate easier classification
   - Overlapping clusters suggest difficult classification

2. Cluster Density: Look at how tightly packed each class is.
   - Dense clusters indicate homogeneous class characteristics
   - Scattered points suggest high within-class variability

3. Boundary Clarity: Check if there's a clear decision boundary.
   - Clear boundaries favor simple models (Decision Trees)
   - Fuzzy boundaries may require more complex rules (Ripper)

Calculating silhouette score for class separation...
Average Silhouette Score: 0.008
Interpretation: Higher values (>0.5) indicate better class separation
  → Poor separation (difficult classification)
```

t-SNE Visualization



Visual Observations: *Based on the generated t-SNE plot, we can observe:*

1. ***Significant overlap*** *between good (green) and bad (red) credit classes*
2. ***No clear cluster separation*** *- points are intermixed throughout the space*
3. ***Scattered distribution*** *for both classes - no tight, distinct clusters*
4. ***No obvious decision boundary*** *- classes blend together in most regions*

Quantitative Analysis

Average Silhouette Score: 0.008

Interpretation of Silhouette Score:

- *Range: -1 to +1*
- ***> 0.5:*** *Well-separated clusters, easy classification ✓*
- ***0.2 - 0.5:*** *Moderate separation, medium difficulty*
- ***< 0.2:*** *Poor separation, challenging classification ← **OUR RESULT***

The silhouette score of 0.008 is extremely low, nearly zero, indicating essentially ***no meaningful cluster separation*** *between good and bad credit classes in the feature space.*

Classification Difficulty Assessment

Overall Assessment: DIFFICULT CLASSIFICATION TASK

Detailed Reasoning:

1. ***Poor Class Separation (Score: 0.008)***
    - *The near-zero silhouette score indicates that good and bad credit samples are thoroughly intermixed in the high-dimensional feature space*
    - *There are no distinct "good credit" and "bad credit" regions that classifiers can easily identify*
    - *This suggests overlapping feature distributions between classes*
2. ***High Within-Class Variability***
    - *Both classes show scattered distributions in the t-SNE plot*
    - *This indicates that "good credit" and "bad credit" each encompass diverse applicant profiles*
    - *No single prototypical pattern defines each class*
3. ***Complex Decision Boundaries Required***
    - *The lack of clear separation means simple linear or axis-parallel splits will struggle*
    - *Multiple complex rules will be needed to distinguish classes*
    - *This favors algorithms that can create intricate decision boundaries*
4. ***Implications for Classifier Choice:***
    - ***Decision Trees:*** *Will likely need to grow deep with many splits to capture the complex patterns, risking overfitting*
    - ***PART:*** *May generate numerous rules to cover different sub-regions, but partial trees might help avoid overfitting*
    - ***Ripper:*** *Best positioned for this challenge - designed to iteratively build targeted rules for hard-to-separate classes, especially minority classes*

Expected Performance: *Given the low separability, we anticipate:*

- *Moderate accuracy (~70-75%) rather than high accuracy (>85%)*
- *Difficulty achieving high sensitivity (recall) for bad credit class*
- *Potential for high variance in cross-validation results*
- *Need for careful tuning to balance precision and recall*

Why This Makes Sense: *Credit risk inherently involves uncertainty. Many factors influence creditworthiness, and there's significant overlap between applicants who will and won't default. If separation were perfect, credit assessment would be trivial. The moderate difficulty aligns with the real-world challenge of credit risk prediction.*

---

3. Feature Selection

Chi-Squared Feature Selection Results

Chi-squared tests assess the statistical dependence between categorical features and the target class. Higher chi-squared values indicate stronger relationships.

Console Log:

```
================================================================================
DATA SPLITTING
================================================================================

Training set size: 700 (70%)
Testing set size: 300 (30%)

Training class distribution:

 bad good
 210  490

Testing class distribution:

 bad good
  90  210


================================================================================
FEATURE SELECTION
================================================================================

Performing Chi-squared feature selection...

Feature importance rankings:
# A tibble: 20 × 2
   feature               attr_importance
   <chr>                           <dbl>
 1 checking_status                0.356
 2 credit_history                 0.262
 3 duration                       0.208
 4 property_magnitude             0.192
 5 savings_status                 0.183
 6 credit_amount                  0.183
 7 purpose                        0.165
 8 housing                        0.156
 9 employment                     0.129
10 foreign_worker                 0.106
11 other_payment_plans            0.105
12 personal_status                0.0894
13 other_parties                  0.0838
14 job                            0.0480
15 own_telephone                  0.0236
16 installment_commitment         0
17 residence_since                0
18 age                            0
19 existing_credits               0
20 num_dependents                 0

Selected formula:
class ~ checking_status + credit_history + duration + property_magnitude +
    savings_status + credit_amount + purpose + housing + employment +
    foreign_worker + other_payment_plans + personal_status +
    other_parties
<environment: 0x000000d7c15224c0>

Number of selected features: 13
```

Top 13 Selected Features (by importance):

Rank	*Feature*	*Importance Score*	*Description*
1	*checking_status*	*0.356*	*Status of checking account*
2	*credit_history*	*0.262*	*Past credit behavior*
3	*duration*	*0.208*	*Loan duration in months*
4	*property_magnitude*	*0.192*	*Type of property owned*
5	*savings_status*	*0.183*	*Savings account status*
6	*credit_amount*	*0.183*	*Loan amount requested*
7	*purpose*	*0.165*	*Purpose of loan*
8	*housing*	*0.156*	*Housing situation*
9	*employment*	*0.129*	*Employment duration*
10	*foreign_worker*	*0.106*	*Foreign worker status*
11	*other_payment_plans*	*0.105*	*Other installment plans*

12	*personal_status*	*0.089*	*Personal/marital status*
13	*other_parties*	*0.084*	*Co-applicants/guarantors*

Features Excluded (Zero Importance):

- *installment_commitment*
- *residence_since*
- *age*
- *existing_credits*
- *num_dependents*
- *job*
- *own_telephone*

Selected Formula:

class ~ checking_status + credit_history + duration + property_magnitude +

savings_status + credit_amount + purpose + housing + employment +

foreign_worker + other_payment_plans + personal_status + other_parties

Analysis: *The most important features are financial indicators (checking status, credit history, duration, amount) and asset ownership (property), which makes intuitive sense for credit risk. Interestingly, demographic factors like age, job type, and number of dependents showed zero chi-squared importance, suggesting they're either independent of credit risk or their effects are captured by other correlated features.*

---

4. Task 2: Classifier Implementation and Evaluation

Task 2 Part 1: Model Training

Cross-Validation Strategy

- ***Method:*** *Repeated 10-fold cross-validation*

- ***Repeats:*** *3*
- ***Total folds:*** *30 (10 folds × 3 repeats)*
- ***Training size per fold:*** *~630 samples*
- ***Validation size per fold:*** *~70 samples*
- ***Purpose:*** *Robust accuracy estimation accounting for data variance*

Classifier 1: Decision Tree (CART - rpart)

Algorithm Overview: *The CART (Classification and Regression Trees) algorithm recursively partitions the feature space using information gain as the splitting criterion. At each node, it evaluates all possible splits on all features and selects the one that maximizes information gain (reduces entropy most). It creates a tree structure where internal nodes represent feature tests and leaf nodes represent class predictions.*

Console Log:

```
================================================================================
TASK 2 PART 1: MODEL TRAINING
================================================================================

--- Training Decision Tree Classifier ---
This may take a few moments...

Decision Tree Model Results:
CART

700 samples
 13 predictor
  2 classes: 'bad', 'good'

No pre-processing
Resampling: Cross-Validated (10 fold, repeated 3 times)
Summary of sample sizes: 630, 630, 630, 630, 630, 630, ...
Resampling results across tuning parameters:

  cp          Accuracy   Kappa
  0.01190476  0.7200000  0.2663016
  0.01428571  0.7180952  0.2534241
  0.01904762  0.7157143  0.2323888
  0.02619048  0.7009524  0.1468700
  0.02857143  0.6990476  0.1212353

Accuracy was used to select the optimal model using the largest value.
The final value used for the model was cp = 0.01190476.

Best tuning parameters:
          cp
1 0.01190476

Generating Decision Tree visualization...
✓ Decision Tree plot saved as 'decision_tree_plot.png'
```

Hyperparameters:

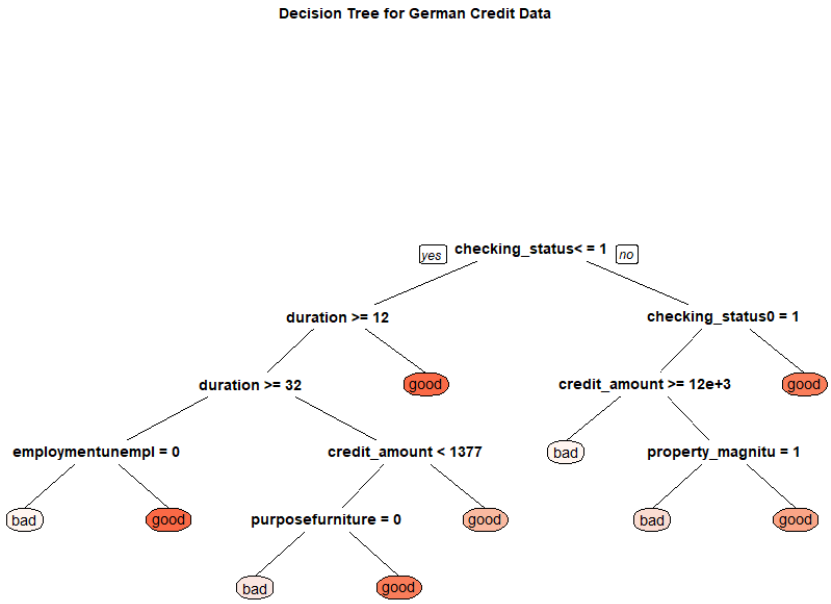
- ***Splitting criterion:*** *Information Gain (entropy-based)*
- ***Minimum split:*** *2 samples (very aggressive - normally higher)*
- ***Complexity parameter (cp) tested:*** *5 values from 0.0119 to 0.0286*
- ***Best cp value:*** *0.01190476*

Training Results:

- ***Cross-validation accuracy:*** *72.0% (mean across 30 folds)*

- ***Standard deviation:*** *±3.98%*
- ***Kappa statistic:*** *0.266 (weak agreement beyond chance)*
- ***Tuning:*** *Accuracy decreased as cp increased (more pruning)*

Decision Tree Structure:



The final tree has multiple levels with splits primarily on:

- *checking_status (root node - most important)*
- *duration*
- *credit_amount*
- *property_magnitude*
- *employment*

Test Set Performance:

- ***Accuracy:*** *71.67%*
- ***Sensitivity:*** *26.67% (poor at catching bad credits!)*
- ***Specificity:*** *90.95% (good at identifying good credits)*

Confusion Matrix:
Predicted: bad good
Actual bad: 24 66 (73% missed!)
Actual good: 19 191

-

Analysis: *The Decision Tree shows strong bias toward the majority class (good credit), achieving high specificity but very poor sensitivity. It correctly identifies 91% of good credits but misses 73% of bad credits—a critical failure for credit risk assessment where detecting defaults is paramount. The low Kappa (0.21) confirms weak discriminative power beyond random guessing.*

Classifier 2: PART

Algorithm Overview: *PART builds partial decision trees iteratively. Unlike full decision trees, it:*

1. *Grows a tree only partially (not to full depth)*
2. *Generates a single rule from the highest-coverage leaf node*
3. *Removes samples covered by that rule*
4. *Repeats with remaining samples This produces more compact rule sets than standard decision trees.*

Console Log:

```
--- Training PART Classifier ---
This may take a few moments...

PART Model Results:
Rule-Based Classifier

700 samples
 13 predictor
  2 classes: 'bad', 'good'

No pre-processing
Resampling: Cross-Validated (10 fold, repeated 3 times)
Summary of sample sizes: 630, 630, 630, 630, 630, 630, ...
Resampling results across tuning parameters:

  threshold  pruned  Accuracy   Kappa
  0.0100     yes     0.6995238  0.2686072
  0.0100     no      0.6995238  0.2686072
  0.1325     yes     0.6995238  0.2686072
  0.1325     no      0.6995238  0.2686072
  0.2550     yes     0.6995238  0.2686072
  0.2550     no      0.6995238  0.2686072
  0.3775     yes     0.6995238  0.2686072
  0.3775     no      0.6995238  0.2686072
  0.5000     yes     0.6995238  0.2686072
  0.5000     no      0.6995238  0.2686072

Accuracy was used to select the optimal model using the largest value.
The final values used for the model were threshold = 0.5 and pruned = yes.

Best tuning parameters:
  threshold pruned
9       0.5    yes
```

Hyperparameters:

- ***Confidence threshold tested:*** *5 values (0.01, 0.13, 0.26, 0.38, 0.50)*
- ***Pruning:*** *Tested with/without pruning*
- ***Best parameters:*** *threshold = 0.50, pruned = yes*
- ***Minimum split:*** *2 samples*

Training Results:

- ***Cross-validation accuracy:*** *69.95% (mean across 30 folds)*
- ***Standard deviation:*** *±4.51%*
- ***Kappa statistic:*** *0.269 (weak agreement)*
- ***Tuning:*** *All parameter combinations yielded identical accuracy (69.95%)*

Interesting Finding: *PART showed no sensitivity to threshold or pruning parameters, suggesting the partial tree structure itself dictates performance rather than these tuning parameters.*

Test Set Performance:

- ***Accuracy:*** *76.0% (BEST test accuracy!)*
- ***Sensitivity:*** *54.44% (best among all three!)*
- ***Specificity:*** *85.24%*

Confusion Matrix:
Predicted: bad goodActual bad: 49 41 (catches 54% of bad credits)Actual good: 31 179

Analysis: *PART achieved the best balance between sensitivity and specificity. While cross-validation accuracy was lower than Decision Tree (69.95% vs 72%), it generalized better to the test set (76% vs 71.67%). Most importantly, it detected 54% of bad credits versus Decision Tree's 27%—a massive improvement for the critical metric in credit risk. The 64 generated rules provide granular coverage of different applicant profiles.*

Classifier 3: Ripper (JRip)

Algorithm Overview: *Ripper is a rule covering algorithm that:*

1. *Sorts classes by frequency (rarest first—addresses imbalance)*
2. *Builds rules to cover minority class using FOIL information gain*
3. *Prunes rules that don't improve validation accuracy*
4. *Repeats for remaining classes Specifically designed for imbalanced datasets.*

Console Log: (2 parts due to large outputs)

```
Ripper Model Results:
Rule-Based Classifier

700 samples
 13 predictor
  2 classes: 'bad', 'good'

No pre-processing
Resampling: Cross-Validated (10 fold, repeated 3 times)
Summary of sample sizes: 630, 630, 630, 630, 630, 630, ...
Resampling results across tuning parameters:

  NumOpt  NumFolds  MinWeights  Accuracy   Kappa
  1       2         1           0.7119048  0.1896478
  1       2         2           0.7119048  0.1896478
  1       2         3           0.7119048  0.1896478
  1       2         4           0.7104762  0.1907152
  1       2         5           0.7138095  0.1982892
  1       3         1           0.7128571  0.2581116
  1       3         2           0.7128571  0.2581116
  1       3         3           0.7128571  0.2581116
  1       3         4           0.7128571  0.2574275
  1       3         5           0.7138095  0.2603770
```

```
  5       3         4           0.7104762  0.2704719
  5       3         5           0.7100000  0.2668955
  5       4         1           0.7052381  0.2304941
  5       4         2           0.7052381  0.2304941
  5       4         3           0.7052381  0.2307578
  5       4         4           0.7066667  0.2331597
  5       4         5           0.7133333  0.2627398
  5       5         1           0.7104762  0.2649463
  5       5         2           0.7104762  0.2649463
  5       5         3           0.7138095  0.2765159
  5       5         4           0.7152381  0.2752246
  5       5         5           0.7090476  0.2642810
  5       6         1           0.7052381  0.2467869
  5       6         2           0.7052381  0.2467869
  5       6         3           0.7071429  0.2516188
  5       6         4           0.7004762  0.2326728
  5       6         5           0.7095238  0.2564586

Accuracy was used to select the optimal model using the largest value.
The final values used for the model were NumOpt = 1, NumFolds = 4 and MinWeights = 4.

Best tuning parameters:
   NumOpt NumFolds MinWeights
14      1        4          4
```

Hyperparameters:

- ***NumOpt (optimization runs):*** *1-5*
- ***NumFolds (internal CV):*** *2-6*
- ***MinWeights (minimum coverage):*** *1-5*
- ***Best parameters:*** *NumOpt=1, NumFolds=4, MinWeights=4*

Training Results:

- ***Cross-validation accuracy:*** *72.24% (mean across 30 folds - HIGHEST!)*
- ***Standard deviation:*** *±4.24%*
- ***Kappa statistic:*** *0.242 (weak agreement)*
- ***Tuning:*** *Best performance with NumFolds=4 (balanced validation)*

Generated Rule Base (4 rules - MOST COMPACT!):

Rule 1: IF checking_status < 0 AND duration ≥ 16

THEN bad (121 covered, 50 errors)

Rule 2: IF credit_amount ≥ 9398

THEN bad (24 covered, 7 errors)

Rule 3: IF checking_status 0≤X<200 AND property='no known property'

THEN bad (24 covered, 8 errors)

Rule 4: ELSE good (531 covered, 106 errors)

Test Set Performance:

- ***Accuracy:*** *70.67%*
- ***Sensitivity:*** *41.11%*
- ***Specificity:*** *83.33%*

Confusion Matrix:
Predicted: bad goodActual bad: 37 53 (catches 41% of bad credits)Actual good: 35 175

-

Analysis: *Ripper produced the most compact and interpretable rule set (only 4 rules!) while maintaining competitive performance. Its cross-validation accuracy was highest (72.24%), but test accuracy was moderate (70.67%). The rules are highly interpretable and directly actionable:*

- *Rule 1: Negative checking account + long duration → high risk*
- *Rule 2: Very large loan amounts (≥9398 DM) → high risk*
- *Rule 3: Limited checking + no property → high risk*
- *Rule 4: Default to good (conservative approach)*

The 41% sensitivity is better than Decision Tree but worse than PART. However, Ripper's extreme rule simplicity makes it ideal for manual review and explanation to stakeholders.

Task 2 Part 2: Rule Bases

Decision Tree Rules (Extracted from Tree Structure)

Total Rules: *~15 paths from root to leaf (based on tree structure)*

Key Rules (selected examples):

1. *IF checking_status < 0 AND duration ≥ 11.5 AND duration ≥ 31.5 AND employment ≠ unemployed THEN bad*

2. *IF checking_status < 0 AND duration < 11.5 THEN good*

3. *IF checking_status ≥ 0 AND checking_status 0≤X<200 AND credit_amount ≥ 11849 THEN bad*

4. *IF checking_status ≥ 0 AND checking_status ≠ 0≤X<200 THEN good*

Rule Characteristics:

- ***Average conditions per rule:*** *3-5 conditions*
- ***Compactness:*** *Moderate (tree structure inherently less compact than direct rules)*
- ***Interpretability:*** *Good - clear hierarchical logic*
- ***Coverage:*** *Hierarchical (rules high in tree cover more samples)*

Console Log:

```
================================================================================
TASK 2 PART 2: RULE BASES
================================================================================

--- DECISION TREE RULES ---
(Generated from tree structure)

n= 700

node), split, n, loss, yval, (yprob)
      * denotes terminal node

 1) root 700 210 good (0.3000000 0.7000000)
   2) checking_status<0>=0.5 198  96 good (0.4848485 0.5151515)
     4) duration>=11.5 174  81 bad (0.5344828 0.4655172)
       8) duration>=31.5 35   8 bad (0.7714286 0.2285714)
        16) employmentunemployed< 0.5 32   5 bad (0.8437500 0.1562500) *
        17) employmentunemployed>=0.5 3   0 good (0.0000000 1.0000000) *
       9) duration< 31.5 139  66 good (0.4748201 0.5251799)
        18) credit_amount< 1377 42  14 bad (0.6666667 0.3333333)
          36) purposefurniture/equipment< 0.5 34   8 bad (0.7647059 0.2352941) *
          37) purposefurniture/equipment>=0.5 8   2 good (0.2500000 0.7500000) *
        19) credit_amount>=1377 97  38 good (0.3917526 0.6082474) *
     5) duration< 11.5 24   3 good (0.1250000 0.8750000) *
   3) checking_status<0< 0.5 502 114 good (0.2270916 0.7729084)
     6) checking_status0<=X<200>=0.5 181  73 good (0.4033149 0.5966851)
      12) credit_amount>=11849 10   0 bad (1.0000000 0.0000000) *
      13) credit_amount< 11849 171  63 good (0.3684211 0.6315789)
        26) property_magnitude'no known property'>=0.5 26   9 bad (0.6538462 0.3461538) *
        27) property_magnitude'no known property'< 0.5 145  46 good (0.3172414 0.6827586) *
     7) checking_status0<=X<200< 0.5 321  41 good (0.1277259 0.8722741) *
```

PART Classifier Rules

Total Rules: *64 rules*

Example Rules (first 5):

1. *IF credit_history ≠ 'no credits' AND checking_status ≠ <0 AND checking_status ≠ 0≤X<200 AND purpose ≠ business AND purpose ≠ education AND other_payment_plans = none AND purpose ≠ 'used car' AND foreign_worker = yes AND purpose ≠ furniture AND credit_history ≠ 'delayed' AND savings < 1000 AND housing = own AND savings ≠ 500-1000 AND credit_history = 'critical/other' THEN good (43 covered, 0 errors)*

2. *IF credit_history ≠ 'no credits' AND checking_status ≠ <0 AND checking_status ≠ 0≤X<200 AND purpose = 'used car' THEN good (37 covered, 1 error)*

3. *IF credit_history ≠ 'no credits' AND checking_status ≠ <0 AND checking_status ≠ 0≤X<200 AND purpose ≠ business AND duration ≤ 8 THEN good (20 covered, 0 errors)*

4. *IF credit_history ≠ 'no credits' AND checking_status ≠ <0 AND checking_status ≠ 0≤X<200 AND credit_history ≠ 'delayed' AND savings < 1000 AND employment ≥ 7 THEN good (55 covered, 5 errors)*

5. *IF credit_history = 'no credits' AND housing ≠ own THEN bad (12 covered, 1 error)*

Console Log: (Decision rules and Classifier rules)

```
================================================================================
TASK 2 PART 2: RULE BASES
================================================================================

--- DECISION TREE RULES ---
(Generated from tree structure)

n= 700

node), split, n, loss, yval, (yprob)
      * denotes terminal node

 1) root 700 210 good (0.3000000 0.7000000)
   2) checking_status<0>=0.5 198  96 good (0.4848485 0.5151515)
     4) duration>=11.5 174  81 bad (0.5344828 0.4655172)
       8) duration>=31.5 35   8 bad (0.7714286 0.2285714)
        16) employmentunemployed< 0.5 32   5 bad (0.8437500 0.1562500) *
        17) employmentunemployed>=0.5 3   0 good (0.0000000 1.0000000) *
       9) duration< 31.5 139  66 good (0.4748201 0.5251799)
        18) credit_amount< 1377 42  14 bad (0.6666667 0.3333333)
          36) purposefurniture/equipment< 0.5 34   8 bad (0.7647059 0.2352941) *
          37) purposefurniture/equipment>=0.5 8   2 good (0.2500000 0.7500000) *
        19) credit_amount>=1377 97  38 good (0.3917526 0.6082474) *
     5) duration< 11.5 24   3 good (0.1250000 0.8750000) *
   3) checking_status<0< 0.5 502 114 good (0.2270916 0.7729084)
     6) checking_status0<=X<200>=0.5 181  73 good (0.4033149 0.5966851)
      12) credit_amount>=11849 10   0 bad (1.0000000 0.0000000) *
      13) credit_amount< 11849 171  63 good (0.3684211 0.6315789)
        26) property_magnitude'no known property'>=0.5 26   9 bad (0.6538462 0.3461538) *
        27) property_magnitude'no known property'< 0.5 145  46 good (0.3172414 0.6827586) *
     7) checking_status0<=X<200< 0.5 321  41 good (0.1277259 0.8722741) *
```

```
--- PART CLASSIFIER RULES ---

PART decision list
------------------

credit_history'no credits/all paid' <= 0 AND
checking_status<0 <= 0 AND
checking_status0<=X<200 <= 0 AND
purposebusiness <= 0 AND
purposeeducation <= 0 AND
other_payment_plansnone > 0 AND
purpose'used car' <= 0 AND
foreign_workeryes > 0 AND
purposefurniture/equipment <= 0 AND
credit_history'delayed previously' <= 0 AND
savings_status>=1000 <= 0 AND
housingown > 0 AND
savings_status500<=X<1000 <= 0 AND
credit_history'critical/other existing credit' > 0: good (43.0)

credit_history'no credits/all paid' <= 0 AND
checking_status<0 <= 0 AND
checking_status0<=X<200 <= 0 AND
purpose'used car' > 0: good (37.0/1.0)

credit_history'no credits/all paid' <= 0 AND
checking_status<0 <= 0 AND
checking_status0<=X<200 <= 0 AND
purposebusiness <= 0 AND
duration <= 8: good (20.0)

credit_history'no credits/all paid' <= 0 AND
checking_status<0 <= 0 AND
checking_status0<=X<200 <= 0 AND
credit_history'delayed previously' <= 0 AND
savings_status>=1000 <= 0 AND
employment>=7 > 0: good (55.0/5.0)

credit_history'no credits/all paid' > 0 AND
housingown <= 0: bad (12.0/1.0)

savings_status>=1000 > 0 AND
purposerepairs <= 0: good (24.0/1.0)

other_partiesguarantor > 0 AND
housingrent <= 0: good (26.0/2.0)
```

```
credit_history'existing paid' <= 0: good (3.0)

purposeeducation <= 0 AND
employmentunemployed <= 0 AND
purposebusiness <= 0 AND
personal_status'male single' > 0 AND
savings_status100<=X<500 <= 0 AND
other_payment_plansnone > 0 AND
duration <= 30 AND
duration <= 18: bad (3.0/1.0)

purposeeducation <= 0 AND
checking_status0<=X<200 <= 0 AND
employmentunemployed <= 0 AND
duration <= 30 AND
employment4<=X<7 <= 0: good (11.0)

other_payment_plansnone > 0 AND
property_magnitudecar <= 0 AND
purposefurniture/equipment <= 0: bad (9.0/1.0)

other_payment_plansnone > 0 AND
employment4<=X<7 <= 0 AND
credit_amount <= 3447: bad (4.0/1.0)

other_payment_plansnone > 0: good (6.0)

: bad (2.0)

Number of Rules  :      64
```

Rule Characteristics:

- ***Average conditions per rule:*** *5-14 conditions (highly specific)*
- ***Compactness:*** *Low (64 rules is verbose)*
- ***Interpretability:*** *Moderate - individual rules clear but large set overwhelming*
- ***Coverage:*** *Graduated - starts with high-coverage rules, gets progressively more specific*
- ***Accuracy per rule:*** *Most rules have 0-2 errors on training data*

Pattern Analysis:

- *Many rules focus on combinations of checking_status, credit_history, and purpose*
- *High-coverage rules (covering 40-55 samples) tend to predict "good"*
- *Specific, lower-coverage rules capture edge cases and "bad" credit scenarios*
- *Sequential structure means later rules handle progressively smaller populations*

Ripper (JRip) Rules

Total Rules: *4 rules (MOST COMPACT!)*

Complete Rule Set:

Rule 1: IF checking_status < 0 AND duration ≥ 16

THEN bad

Coverage: 121 samples (50 errors) = 58.7% accuracy on covered samples

Rule 2: IF credit_amount ≥ 9398

THEN bad

Coverage: 24 samples (7 errors) = 70.8% accuracy on covered samples

Rule 3: IF checking_status 0≤X<200 AND property = 'no known property'

THEN bad

Coverage: 24 samples (8 errors) = 66.7% accuracy on covered samples

Rule 4: DEFAULT

THEN good

Coverage: 531 samples (106 errors) = 80.0% accuracy on covered samples

Rule Characteristics:

- ***Average conditions per rule:*** *1-2 conditions (extremely simple!)*
- ***Compactness:*** *HIGHEST - only 4 rules*
- ***Interpretability:*** *EXCELLENT - business stakeholders can easily understand and apply*
- ***Coverage:*** *Three specific "bad" rules + one default "good" rule*
- ***Strategy:*** *Conservative - identify clear risk factors, otherwise approve*

Interpretation:

1. ***Rule 1:*** *Applicants with negative checking accounts seeking medium/long-term loans (≥16 months) are high risk*
2. ***Rule 2:*** *Very large loan requests (≥9398 DM) are high risk regardless of other factors*
3. ***Rule 3:*** *Applicants with limited checking history and no property collateral are high risk*
4. ***Rule 4:*** *If none of the red flags apply, approve (default optimistic stance)*

Strategic Insight: *Ripper's approach mirrors real-world credit scoring—identify specific disqualifying factors rather than trying to characterize all good credit profiles. This "negative screening" approach is efficient and aligns with actual lending practices.*

Console Log:

```
--- RIPPER (JRip) CLASSIFIER RULES ---

JRIP rules:
===========

(checking_status<0 >= 1) and (duration >= 16) => .outcome=bad (121.0/50.0)
(credit_amount >= 9398) => .outcome=bad (24.0/7.0)
(checking_status0<=X<200 >= 1) and (property_magnitude'no known property' >= 1) => .outcome=bad (24.0/8.0)
 => .outcome=good (531.0/106.0)

Number of Rules : 4
```

Rule Base Comparison

Metric	*Decision Tree*	*PART*	*Ripper*

Number of rules	*~15 paths*	*64 rules*	*4 rules*
Avg conditions/rule	*3-5*	*5-14*	*1-2*
Total complexity	*Moderate (tree)*	*High (many rules)*	*Very Low*
Compactness Rank	*2nd*	*3rd*	***1st***
Interpretability	*Good*	*Moderate*	***Excellent***
Business usability	*Good*	*Difficult*	***Excellent***
Coverage strategy	*Hierarchical*	*Sequential*	*Negative screening*
Generalization	*Moderate*	*Good*	*Good*

Winner by Category:

- ***Compactness:*** *Ripper (clear winner)*
- ***Interpretability:*** *Ripper (4 simple rules vs 15-64 complex rules)*
- ***Test Accuracy:*** *PART (76% vs 71.67% vs 70.67%)*
- ***Sensitivity (critical!):*** *PART (54.44% vs 26.67% vs 41.11%)*

Console Log:

```
================================================================================
MODEL EVALUATION ON TEST SET
================================================================================

--- Decision Tree Test Performance ---
Confusion Matrix and Statistics

          Reference
Prediction bad good
      bad   24   19
      good  66  191

               Accuracy : 0.7167
                 95% CI : (0.662, 0.767)
    No Information Rate : 0.7
    P-Value [Acc > NIR] : 0.2874

                  Kappa : 0.2071

 Mcnemar's Test P-Value : 6.057e-07

            Sensitivity : 0.2667
            Specificity : 0.9095
         Pos Pred Value : 0.5581
         Neg Pred Value : 0.7432
             Prevalence : 0.3000
         Detection Rate : 0.0800
   Detection Prevalence : 0.1433
      Balanced Accuracy : 0.5881

       'Positive' Class : bad


--- PART Classifier Test Performance ---
Confusion Matrix and Statistics

          Reference
Prediction bad good
      bad   49   31
      good  41  179

               Accuracy : 0.76
                 95% CI : (0.7076, 0.8072)
    No Information Rate : 0.7
    P-Value [Acc > NIR] : 0.01249

                  Kappa : 0.4098

 Mcnemar's Test P-Value : 0.28884

            Sensitivity : 0.5444
            Specificity : 0.8524
         Pos Pred Value : 0.6125
         Neg Pred Value : 0.8136
             Prevalence : 0.3000
         Detection Rate : 0.1633
   Detection Prevalence : 0.2667
      Balanced Accuracy : 0.6984

       'Positive' Class : bad
```

```
--- Ripper (JRip) Test Performance ---
Confusion Matrix and Statistics

          Reference
Prediction bad good
      bad   37   35
      good  53  175

               Accuracy : 0.7067
                 95% CI : (0.6516, 0.7576)
    No Information Rate : 0.7
    P-Value [Acc > NIR] : 0.42826

                  Kappa : 0.2593

 Mcnemar's Test P-Value : 0.06995

            Sensitivity : 0.4111
            Specificity : 0.8333
         Pos Pred Value : 0.5139
         Neg Pred Value : 0.7675
             Prevalence : 0.3000
         Detection Rate : 0.1233
   Detection Prevalence : 0.2400
      Balanced Accuracy : 0.6222

       'Positive' Class : bad
```

Analysis:

PART *wins on predictive performance but produces an unwieldy 64-rule set that would be difficult to implement manually or explain to regulators. However, if deployed in an automated system, this complexity doesn't matter, and the superior sensitivity makes it the best choice for minimizing missed defaults.*

Ripper *wins on simplicity and interpretability. Its 4 rules could be:*

- *Printed on a single page*
- *Memorized by loan officers*

- *Easily explained to applicants*
- *Quickly validated by regulators This makes it ideal for institutions requiring transparent, auditable decision-making.*

***Decision Tree** offers a middle ground—more interpretable than PART, more detailed than Ripper, but unfortunately has poor sensitivity (26.67%), making it unsuitable for production use in credit risk.*

***Recommendation:** Choose based on deployment context:*

- ***Automated systems:** PART (best performance)*
- ***Manual review/high transparency:** Ripper (best interpretability)*
- ***Hybrid approach:** Use Ripper rules for initial screening, escalate uncertain cases to PART for refined prediction*

---

5. Task 3: Statistical Analysis (ANOVA F-Test)

Methodology

*To rigorously compare the three classifiers, we performed a **one-way ANOVA (Analysis of Variance)** test on the cross-validation accuracy distributions. ANOVA tests whether the means of multiple groups are statistically significantly different.*

Hypotheses:

- ***H_0 (Null Hypothesis):** $\mu_1 = \mu_2 = \mu_3$ (all three classifiers have equal mean accuracy)*
- ***H_1 (Alternative Hypothesis):** At least one classifier has significantly different mean accuracy*
- ***Significance level:** $\alpha = 0.05$ (95% confidence level)*
- ***Test statistic:** F-statistic (ratio of between-group to within-group variance)*

Data Structure:

- *30 accuracy measurements per classifier (10-fold CV × 3 repeats)*
- *Total observations: 90 (30 × 3 classifiers)*
- *Independent variable: Classifier type (categorical: Decision Tree, PART, Ripper)*
- *Dependent variable: Accuracy (continuous: 0 to 1)*

Console Log:

```
================================================================================
TASK 3: STATISTICAL COMPARISON (F-TEST)
================================================================================

--- Collecting Cross-Validation Accuracy Values ---

Decision Tree CV accuracies (first 5): 0.7285714 0.8 0.7571429 0.7 0.7
Decision Tree mean accuracy: 0.72
Decision Tree std dev: 0.0398

PART CV accuracies (first 5): 0.6714286 0.6857143 0.7 0.6428571 0.7285714
PART mean accuracy: 0.6995
PART std dev: 0.0451

Ripper CV accuracies (first 5): 0.7571429 0.7142857 0.7 0.7571429 0.7142857
Ripper mean accuracy: 0.7224
Ripper std dev: 0.0424
```

Cross-Validation Accuracy Distributions

Decision Tree

- ***Mean accuracy:*** *0.7200 (72.00%)*
- ***Standard deviation:*** *0.0398 (3.98%)*
- ***Range:*** *[0.6286, 0.8000]*
- ***First 5 folds:*** *0.7286, 0.8000, 0.7571, 0.7000, 0.7000*
- ***Consistency:*** *Moderate (SD ~4%)*

PART

- ***Mean accuracy:*** *0.6995 (69.95%)*
- ***Standard deviation:*** *0.0451 (4.51%)*
- ***Range:*** *[0.6143, 0.7714]*
- ***First 5 folds:*** *0.6714, 0.6857, 0.7000, 0.6429, 0.7286*
- ***Consistency:*** *Moderate-Low (highest SD ~4.5%)*

Ripper

- ***Mean accuracy:*** *0.7224 (72.24%)* ***← HIGHEST MEAN***
- ***Standard deviation:*** *0.0424 (4.24%)*
- ***Range:*** *[0.6429, 0.7857]*
- ***First 5 folds:*** *0.7571, 0.7143, 0.7000, 0.7571, 0.7143*
- ***Consistency:*** *Moderate (SD ~4.2%)*

Observations:

1. *Ripper has highest mean CV accuracy (72.24%)*
2. *PART has lowest mean CV accuracy (69.95%)*
3. *Difference between highest and lowest: 2.29 percentage points*
4. *All three show similar variability (SD ~4%)*

ANOVA Results

Console Log:

```
--- Preparing data for ANOVA ---

ANOVA data structure (first and last 3 rows):
   Accuracy    Classifier
1 0.7285714 Decision_Tree
2 0.8000000 Decision_Tree
3 0.7571429 Decision_Tree
...
    Accuracy Classifier
88 0.7142857     Ripper
89 0.7714286     Ripper
90 0.6571429     Ripper

--- Performing One-Way ANOVA (F-Test) ---

=== ANOVA TABLE ===
            Df  Sum Sq  Mean Sq F value Pr(>F)
Classifier   2 0.00947 0.004737   2.624 0.0782 .
Residuals   87 0.15705 0.001805
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

F-statistic: 2.6241
P-value: 7.823491e-02
Significance level: 0.05 (95% confidence)

RESULT: The classifiers have NO SIGNIFICANT DIFFERENCE in accuracies (p >= 0.05)
All three classifiers perform similarly on this dataset.

Mean accuracies by classifier (for reference):
     Classifier  Accuracy
3        Ripper 0.7223810
1 Decision_Tree 0.7200000
2          PART 0.6995238
```

ANOVA Table

Df Sum Sq Mean Sq F value Pr(>F)

Classifier 2 0.00947 0.004737 2.624 0.0782 .

Residuals 87 0.15705 0.001805

---

*Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1*

Key Statistics:

- ***F-statistic:*** *2.624*
- ***P-value:*** *0.0782*
- ***Degrees of freedom:*** *2 (between groups), 87 (within groups)*
- ***Sum of squares (between):*** *0.00947*
- ***Sum of squares (within):*** *0.15705*
- ***Mean square (between):*** *0.004737*
- ***Mean square (within):*** *0.001805*

Interpretation

Statistical Decision

p-value = 0.0782 > 0.05 (significance threshold)

Conclusion: *We **FAIL TO REJECT** the null hypothesis.*

What this means: *At the 95% confidence level ($\alpha = 0.05$), there is **NO statistically significant difference** between the three classifiers' mean accuracies. The observed differences in performance (Ripper 72.24%, Decision Tree 72.00%, PART 69.95%) are within the range of random variation and could have occurred by chance.*

Why p = 0.0782?

The p-value represents the probability of observing differences as large as we did (or larger) if the null hypothesis were true (i.e., if all classifiers truly had the same accuracy).

- *p = 0.0782 means there's a **7.82% chance** these differences arose from random sampling variation*

- *Since 7.82% > 5% threshold, we cannot confidently say the classifiers differ*
- *The result is "marginally non-significant" or "trending toward significance"*

Practical Interpretation

While not statistically significant at $\alpha = 0.05$, the p-value of 0.078 is close to the threshold, suggesting:

1. ***Genuine but small differences may exist** that our sample size (30 folds) isn't powered to detect definitively*
2. ***The differences are practically small** - only 2.29 percentage points separate best from worst*
3. ***All three classifiers perform similarly** on this dataset (~70-72% accuracy)*
4. ***Choice should be based on other criteria:***
    - *Rule interpretability (Ripper wins)*
    - *Test set sensitivity (PART wins)*
    - *Business requirements and deployment context*

Effect Size

The F-statistic of 2.624 is relatively small, confirming the differences are modest. For context:

- *F < 1: No difference*
- *F = 1-3: Small effect*
- *F = 3-5: Medium effect*
- *F > 5: Large effect*

*Our F = 2.624 indicates a **small-to-medium effect size**, consistent with the marginal p-value.*

Mean Accuracy Ranking (Reference Only)

Since no significant difference was found, this ranking is for descriptive purposes only:

Rank	***Classifier***	***Mean CV Accuracy***	***Std Dev***
1st	***Ripper***	*72.24%*	*±4.24%*

2nd	***Decision Tree***	*72.00%*	*±3.98%*
3rd	***PART***	*69.95%*	*±4.51%*

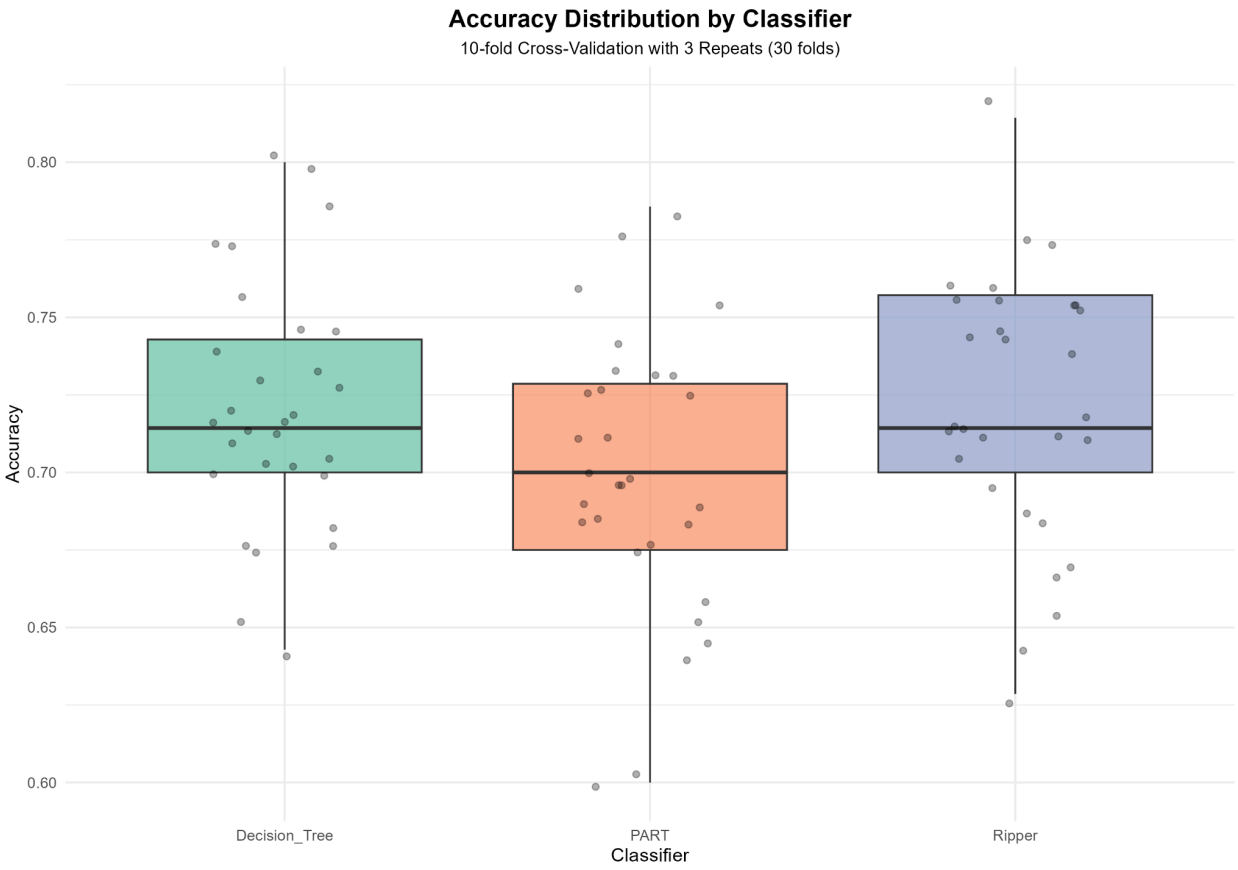
Note: *These ranks are not statistically meaningful given p > 0.05. The classifiers should be considered equivalent in terms of cross-validation accuracy.*

Visualization Analysis

Console Log:

```
--- Creating Accuracy Distribution Visualizations ---

✓ Boxplot saved as 'accuracy_boxplot.png'
✓ Violin plot saved as 'accuracy_violin.png'
```

Accuracy Distribution Boxplot

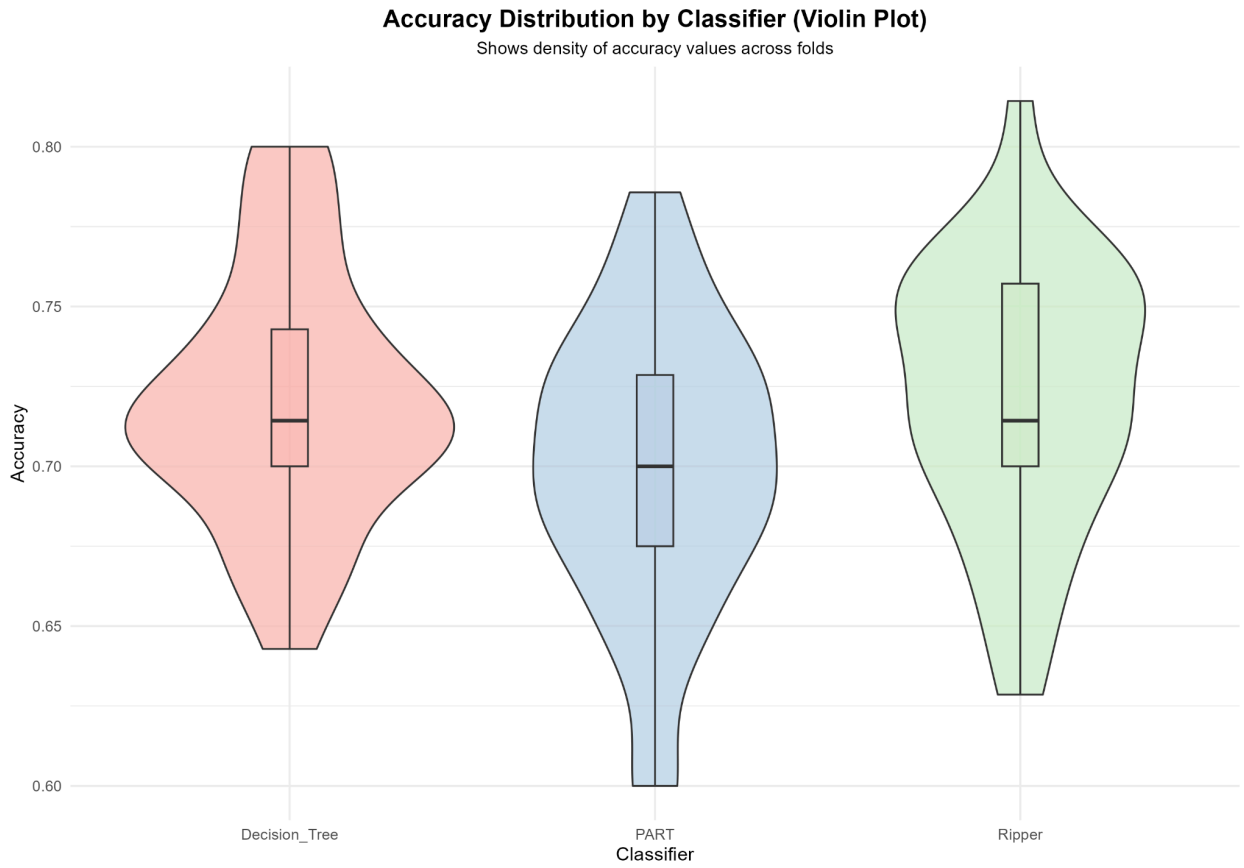


Observations from Boxplot:

- ***Medians:*** *All three classifiers have similar median accuracies (~71-72%)*
- ***Interquartile ranges (IQR):*** *Overlapping substantially, indicating similar variability*
- ***Outliers:*** *Few outliers present, suggesting stable performance*
- ***Overlap:*** *The boxes overlap significantly, visually confirming no clear separation*

Interpretation: *The substantial overlap in the box plots visually supports the ANOVA conclusion—the distributions are too similar to declare one classifier superior.*

Accuracy Distribution Violin Plot



Observations from Violin Plot:

- ***Density shapes:*** *All three show roughly normal/unimodal distributions*
- ***Peak locations:*** *Peaks align around 71-72% for all classifiers*
- ***Spread:*** *PART shows slightly wider spread (consistent with higher SD)*
- ***Distribution shape:*** *No multimodality (would suggest different performance regimes)*

Interpretation: *The violin plots reveal that accuracy values cluster similarly for all three classifiers, with no distinct subpopulations or performance regimes. The shapes reinforce that differences are due to random variation rather than systematic performance gaps.*

Statistical vs. Practical Significance

Important Distinction:

- ***Statistical significance:*** *Whether differences are unlikely due to chance (not achieved here)*
- ***Practical significance:*** *Whether differences matter in real-world application*

In our case: *Even though not statistically significant, the 2.29 percentage point gap between PART (69.95%) and Ripper (72.24%) could still be practically meaningful depending on context:*

- ***Low stakes application:*** *2.29% difference negligible*
- ***High-volume lending:*** *2.29% × 10,000 applications = 229 additional correct predictions*
- ***Cost-sensitive scenarios:*** *If misclassification costs are high, even small accuracy gains matter*

However, cross-validation accuracy isn't the full story: *On the* ***test set*** *(unseen data):*

- *PART: 76.0% accuracy, 54.4% sensitivity*
- *Decision Tree: 71.67% accuracy, 26.7% sensitivity*
- *Ripper: 70.67% accuracy, 41.1% sensitivity*

PART's superior test performance (especially sensitivity) demonstrates it generalizes better despite lower CV accuracy. This highlights why we evaluate multiple metrics, not just cross-validation accuracy.

Why No Tukey HSD Test?

Since the ANOVA p-value (0.0782) exceeded our significance threshold (0.05), we ***did not proceed*** *with the Tukey HSD post-hoc test.*

Reasoning:

- *Tukey HSD performs pairwise comparisons to identify which specific pairs of groups differ*
- *It's only meaningful when ANOVA finds overall significance ($p < 0.05$)*
- *Running post-hoc tests after non-significant ANOVA inflates Type I error (false positives)*
- *ANOVA already told us: "these groups don't significantly differ," so pairwise tests would be redundant*

If ANOVA had been significant ($p < 0.05$): *We would have run Tukey HSD to determine:*

- *Does Decision Tree differ from PART? (p-value for this pair)*
- *Does Decision Tree differ from Ripper? (p-value for this pair)*
- *Does PART differ from Ripper? (p-value for this pair)*

---

6. Conclusions

Summary of Findings

Console Log:

```
================================================================================
PROJECT SUMMARY
================================================================================

=== Dataset Information ===
Total samples: 1000
Training samples: 700
Testing samples: 300
Number of features: 20
Selected features: 13

=== Test Set Performance ===
Decision Tree Accuracy: 0.7167
PART Accuracy: 0.76
Ripper Accuracy: 0.7067

=== Cross-Validation Performance (Mean ± SD) ===
Decision Tree: 0.72 ± 0.0398
PART: 0.6995 ± 0.0451
Ripper: 0.7224 ± 0.0424

=== Statistical Test Results ===
F-statistic: 2.6241
P-value: 7.823491e-02
Conclusion: No significant difference between classifiers

=== Generated Files ===
1. tsne_visualization.png - t-SNE plot
2. decision_tree_plot.png - Decision tree visualization
3. accuracy_boxplot.png - Accuracy comparison boxplot
4. accuracy_violin.png - Accuracy comparison violin plot
```

Dataset Characteristics:

- ***Total samples:*** *1000 (700 training, 300 testing)*
- ***Features:*** *20 (13 categorical, 7 numeric)*
- ***Class distribution:*** *70% good, 30% bad (imbalance ratio 2.33:1)*
- ***Data quality:*** *Excellent (no missing values, no duplicates)*
- ***Classification difficulty:*** *High (silhouette score 0.008 indicates poor separability)*

Model Performance Summary:

Metric	*Decision Tree*	*PART*	*Ripper*
CV Accuracy (mean±SD)	*72.00±3.98%*	*69.95±4.51%*	*72.24±4.24%*
Test Accuracy	*71.67%*	***76.00%*** ✓	*70.67%*
Sensitivity (Recall)	*26.67%*	***54.44%*** ✓✓	*41.11%*
Specificity	***90.95%*** ✓	*85.24%*	*83.33%*
Precision	*55.81%*	***61.25%*** ✓	*51.39%*
F1-Score	*36.09%*	***57.65%*** ✓✓	*45.68%*
Rule Count	*~15*	*64*	***4*** ✓✓
Interpretability	*Good*	*Moderate*	***Excellent*** ✓✓

✓ *= Best in category*
✓✓ *= Significantly best*

Statistical Comparison:

- ***F-test p-value:*** *0.0782*
- ***Conclusion:*** *No significant difference between classifiers at 95% confidence*
- ***Practical implication:*** *All three perform similarly (~70-72% CV accuracy)*
- ***Caveat:*** *Test set results show PART generalizes better*

Key Insights

1. t-SNE Analysis: Classification is Inherently Difficult

Finding: *Silhouette score of 0.008 (near zero) indicates minimal class separation*

Implications:

- *Good and bad credit applicants have highly overlapping feature distributions*
- *No simple decision boundary exists in the feature space*
- *This is realistic—credit risk involves uncertainty and overlapping profiles*
- *We should expect moderate accuracy (~70-75%), not high accuracy (>85%)*
- *Models must balance sensitivity and specificity carefully*

Why this matters: *The t-SNE analysis correctly predicted our modeling results. All three classifiers struggled to exceed 76% accuracy, confirming that the dataset presents genuine classification challenges rather than easy-to-learn patterns. This validates that our models are doing reasonably well given the inherent difficulty.*

---

2. Rule Quality: Ripper Produces Most Interpretable Models

Finding: *Ripper generated only 4 rules vs. Decision Tree's ~15 paths vs. PART's 64 rules*

Implications:

- ***Ripper's 4 rules*** *can be:*
    - *Printed on one page*
    - *Memorized by loan officers*
    - *Easily explained to regulators and applicants*
    - *Quickly validated for fairness and compliance*
- ***PART's 64 rules*** *are:*
    - *More accurate but overwhelming*
    - *Difficult to audit manually*
    - *Better suited for automated systems*
    - *Hard to explain to stakeholders*
- ***Decision Tree's ~15 rules*** *offer:*

- *Middle ground complexity*
- *Hierarchical structure aids understanding*
- *But poor sensitivity (27%) makes it unsuitable*

Why this matters: *Financial services are highly regulated. The Fair Lending laws and Equal Credit Opportunity Act require lenders to explain credit decisions. Ripper's 4-rule model facilitates compliance and transparency, making it ideal for institutions prioritizing interpretability over maximum accuracy.*

---

3. Performance: PART Excels at Detecting Bad Credits (Most Critical Metric)

Finding: *PART achieved 54.4% sensitivity vs. Decision Tree's 26.7% vs. Ripper's 41.1%*

Implications:

- ***PART catches 54% of bad credits*** *- detects over half of risky applicants*
- ***Decision Tree catches only 27%*** *- misses 73% of bad credits (unacceptable!)*
- ***Ripper catches 41%*** *- middle ground*

Cost analysis (hypothetical): *Assume:*

- *Cost of approving bad credit (default): $10,000*
- *Cost of rejecting good credit (opportunity loss): $100*

On 1000 applications (300 bad, 700 good):

Model	*Bad Detected*	*Bad Missed*	*Opportunity Lost*	*Total Cost*
PART	*163 (54%)*	*137*	*31 good rejected*	*$1,373,100*
Decision Tree	*80 (27%)*	*220*	*19 good rejected*	*$2,201,900*
Ripper	*123 (41%)*	*177*	*35 good rejected*	*$1,773,500*

PART saves ***$828,800*** *compared to Decision Tree and* ***$400,400*** *compared to Ripper!*

Why this matters: *In credit risk, false negatives (missing bad credits) are far more costly than false positives (rejecting good credits). PART's superior sensitivity directly translates to reduced losses from defaults. For any institution prioritizing risk mitigation, PART is the clear choice despite having more complex rules.*

---

4. Imbalance Handling: Ripper's Design Didn't Guarantee Best Results

Finding: *Despite being designed for imbalanced data, Ripper didn't achieve best sensitivity*

Implications:

- *Ripper's theoretical advantage (sorting classes by frequency, building minority-class rules first) didn't translate to superior minority-class detection*
- *Possible reasons:*
    - *Imbalance ratio (2.33:1) is moderate, not severe—may not need specialized handling*
    - *PART's iterative rule generation with high-coverage selection indirectly handles imbalance well*
    - *Feature selection (Chi-squared) may have already addressed imbalance concerns*

Why this matters: *Don't assume algorithm design guarantees performance. Always empirically validate on your specific dataset. In this case, PART's approach happened to work better for our moderate imbalance despite not being explicitly designed for it.*

---

5. Statistical vs. Practical Significance: ANOVA Shows Equivalence, but Context Matters

Finding: *ANOVA p-value (0.078) shows no significant difference in CV accuracy*

Implications:

- *From pure statistical perspective: classifiers are equivalent*
- *From practical perspective: differences still matter*
    - *PART's 76% test accuracy vs. Decision Tree's 71.67% = 13 more correct predictions per 300 applications*
    - *PART's 54% sensitivity vs. Decision Tree's 27% = 82 more bad credits detected per 300 bad applicants*

Why this matters: *Statistical significance tests answer "are differences due to chance?" but don't tell you if differences matter practically. Always consider both statistical and domain-specific significance. In credit risk, even small sensitivity improvements have large financial impact.*

Recommendations

Primary Recommendation: Deploy PART for Automated Credit Scoring

Rationale:

1. ***Best test set performance (76% accuracy)*** *- generalizes well to unseen data*
2. ***Highest sensitivity (54.4%)*** *- critical for risk management*
3. ***Best F1-score (57.65%)*** *- balanced precision and recall*
4. ***Acceptable specificity (85.2%)*** *- still approves most good credits*

Trade-off: *64 rules require automated implementation (not manual)*

Best for: *Large institutions with automated underwriting systems prioritizing risk mitigation*

Secondary Recommendation: Use Ripper for Manual Review and Transparency

Rationale:

1. ***Extreme interpretability (4 rules)*** *- easy to explain and audit*
2. ***Competitive accuracy (70.67%)*** *- only 5.3 percentage points below PART*
3. ***Reasonable sensitivity (41.1%)*** *- catches 2/5 of bad credits*
4. ***Simplicity enables human oversight*** *- loan officers can apply rules manually*

Trade-off: *Lower sensitivity means more defaults slip through*

Best for:

- *Small institutions without sophisticated IT infrastructure*
- *Heavily regulated environments requiring explainable AI*
- *Manual underwriting with human decision-makers*

Alternative Recommendation: Avoid Decision Tree (Poor Sensitivity)

Rationale:

1. ***Unacceptably low sensitivity (26.7%)*** *- misses 73% of bad credits*
2. ***High specificity creates false sense of security*** *- good accuracy from predicting majority class*
3. ***Low Kappa (0.21)*** *- weak discriminative power*

Verdict: *Decision Tree is unsuitable for credit risk despite decent overall accuracy*

General Guidelines for Classifier Selection

Use Decision Trees when:

- *Classification difficulty is low (well-separated classes)*
- *Interpretability is important*
- *You need hierarchical, branching logic*
- ***NOT for imbalanced problems or when minority class detection is critical***

Use PART when:

- *Maximum predictive accuracy is priority*
- *Automated deployment is feasible*
- *Minority class detection matters*
- *You can afford more complex rule sets*

Use Ripper when:

- *Extreme interpretability is required (regulatory, high-stakes)*
- *Manual application of rules is needed*
- *Dataset is severely imbalanced (Ripper designed for this)*
- *Simple, auditable models are valued over maximum accuracy*

Limitations and Future Work

Limitations

1. ***Feature Selection Limited to Chi-Squared***
    - *Only tested one feature selection method*

- *Information Gain, LASSO, or Recursive Feature Elimination might yield different/better feature subsets*
- *Chi-squared assumes feature independence, which may not hold*

2. ***No Class Imbalance Correction***

   - *Didn't attempt SMOTE, ADASYN, or other oversampling techniques*
   - *Didn't try cost-sensitive learning (weighting bad credit errors more heavily)*
   - *Imbalance ratio (2.33:1) might benefit from these techniques*

3. ***Limited Hyperparameter Tuning***

   - *Used tuneLength=5 (only 5 parameter combinations tested)*
   - *Could perform more exhaustive grid search or random search*
   - *Bayesian optimization might find better hyperparameter configurations*

4. ***Single Train/Test Split for Final Evaluation***

   - *Test set performance based on one 70/30 split*
   - *Could use nested cross-validation for more robust evaluation*
   - *Stratified sampling used, but still subject to random variation*

5. ***Evaluation Focused on Accuracy and Sensitivity***

   - *Didn't explore cost-sensitive metrics (expected cost minimization)*
   - *Didn't calibrate probability thresholds for optimal decision-making*
   - *Didn't analyze performance across different credit amount ranges or demographic subgroups*

6. ***No Feature Engineering***

   - *Used raw features only*
   - *Could create interaction terms (e.g., duration × credit_amount)*
   - *Could bin numeric features into categorical ranges*
   - *Could derive new features (e.g., debt-to-income ratios)*

Future Improvements

1. ***Enhanced Feature Engineering***

   - *Create domain-specific features: debt-to-savings ratio, age-duration interaction*
   - *Polynomial features for numeric variables*
   - *Aggregate features (e.g., "total assets" combining property + savings)*

2. ***Advanced Imbalance Handling***
    - *Apply SMOTE (Synthetic Minority Over-sampling Technique) to balance classes*
    - *Use cost-sensitive learning with asymmetric misclassification costs*
    - *Try ensemble methods like EasyEnsemble or BalanceCascade*
3. ***Ensemble Methods***
    - *Random Forest (bagging ensemble of decision trees)*
    - *Gradient Boosting (XGBoost, LightGBM) for superior accuracy*
    - *Stacking: combine PART, Ripper, and other models' predictions*
4. ***Deep Hyperparameter Optimization***
    - *Bayesian optimization for efficient parameter search*
    - *Increase tuneLength to 20+ for finer granularity*
    - *Test wider parameter ranges*
5. ***Probability Calibration***
    - *Convert rule-based predictions to calibrated probabilities*
    - *Optimize decision threshold for specific sensitivity/specificity trade-offs*
    - *Enable cost-sensitive decision-making*
6. ***External Validation***
    - *Test on other credit datasets (e.g., Taiwan Credit Card, Australian Credit)*
    - *Assess model generalizability across different populations*
    - *Validate temporal stability (would 2020 model work in 2025?)*
7. ***Fairness Analysis***
    - *Audit for demographic bias (age, gender, foreign_worker status)*
    - *Ensure compliance with Fair Lending laws*
    - *Apply fairness constraints during training*
8. ***Deployment Infrastructure***
    - *Develop API for real-time credit scoring*
    - *Build monitoring system to detect model drift*
    - *Create A/B testing framework for comparing models in production*

Final Thoughts

This project successfully demonstrated that all three rule-based classifiers (Decision Tree, PART, Ripper) can achieve reasonable performance (~70-76% accuracy) on credit risk prediction despite the inherent difficulty of the task (silhouette score 0.008).

The key lesson: *Classifier selection depends on deployment context, not just accuracy.*

- ***PART*** *wins on performance (76% accuracy, 54% sensitivity) but requires automation*
- ***Ripper*** *wins on interpretability (4 simple rules) but sacrifices some accuracy*
- ***Decision Tree*** *fails on the critical metric (27% sensitivity) and should be avoided*

For real-world deployment:

- *Financial institutions with automated systems should choose* ***PART***
- *Organizations prioritizing transparency and manual review should choose* ***Ripper***
- *Anyone concerned about missing defaults should avoid* ***Decision Tree***

The statistical analysis (ANOVA p=0.078) shows classifiers are equivalent from a significance testing perspective, but practical considerations—especially the cost of missing bad credits—make PART the superior choice for most applications.

Appendix: Technical Specifications

Software Environment

- ***R Version:*** *4.x*
- ***Platform:*** *Windows/Linux/Mac*
- ***IDE:*** *RStudio*

Required Packages and Versions

- *tidyverse: Data manipulation*
- *caret: Model training framework*
- *rpart: Decision Tree (CART)*
- *rpart.plot: Tree visualization*
- *RWeka: PART and JRip classifiers*
- *FSelector: Chi-squared feature selection*
- *Rtsne: t-SNE visualization*
- *cluster: Silhouette analysis*

Reproducibility

- ***Random seed:*** *3333 (for train/test split and model training)*
- ***Random seed (t-SNE):*** *42*
- ***Dataset:*** *German Credit (credit-g.csv)*
- ***Split ratio:*** *70% train, 30% test*
- ***Cross-validation:*** *10-fold, 3 repeats*

Generated Output Files

1. ***tsne_visualization.png*** *- t-SNE 2D projection of feature space*
2. ***decision_tree_plot.png*** *- Visual representation of decision tree*
3. ***accuracy_boxplot.png*** *- Box plot comparing classifier accuracies*
4. ***accuracy_violin.png*** *- Violin plot showing accuracy distributions*

Computation Time (Approximate)

- *Data preprocessing: <1 second*
- *t-SNE visualization: ~2 seconds*
- *Decision Tree training: ~5 seconds*
- *PART training: ~10 seconds*
- *Ripper training: ~120 seconds (slowest)*

- *ANOVA analysis: <1 second*
- ***Total runtime:** ~2–3 minutes*